

AI-ML Based Intelligent Dead Reckoning System

Complete Backend Architecture — Flutter/Dart Frontend, Python Backend, Offline-Resilient, Cache-Enabled

This document describes the full backend architecture for a phone-based (Flutter/Dart) dead reckoning navigation system. All processing — sensor fusion (Kalman Filter) and ML-based drift correction — runs on the Python backend. The design accounts for network disconnection, data backlog recovery, and multi-layer caching.

1. Core Concept

Dead reckoning estimates current position using only motion data (no GPS) — similar to estimating your location by counting steps and turns with your eyes closed. The phone's accelerometer, gyroscope, and magnetometer provide this motion data. Because sensor readings contain small errors, repeatedly integrating them to compute position causes accumulating error, known as **drift**. This system reduces drift in two stages: a Kalman Filter (statistical sensor fusion) followed by an ML model (learned drift correction).

2. Full System Architecture

```
FLUTTER APP (Dart)
+-- Sensor Layer (sensors_plus package)
|   reads accelerometer, gyroscope, magnetometer
+-- Local Cache / Offline Buffer (sqflite or Hive)
|   - stores every reading locally FIRST, before sending
|   - also stores last known corrected position (for instant UI display)
+-- Local Fallback Kalman Filter (lightweight, runs on-device)
|   - ONLY active when offline
|   - keeps producing a live (uncorrected) position estimate from
|       buffered sensor data, so the map keeps moving instead of freezing
|   - clearly marked "estimated" on the UI until backend correction returns
+-- Connectivity Monitor (connectivity_plus package)
|   detects online/offline transitions
+-- Sync Manager
|   ONLINE -> streams live + buffered data to backend, uses backend's
|       corrected position once it arrives (overrides local estimate)
|   OFFLINE -> keeps buffering, runs local fallback filter, shows
|       "offline mode - estimated position" indicator
|   RECONNECT -> flushes buffer to backend in timestamp order
+-- WebSocket Client (web_socket_channel package)
    auto-reconnect with exponential backoff (1s, 2s, 4s, 8s...)

    | WebSocket (primary) / REST (fallback)
    v

PYTHON BACKEND (FastAPI)
+-- /ws/track/{device_id} -- WebSocket endpoint, live ingestion
+-- /sensor-data (REST)    -- fallback batch upload endpoint
+-- Ingestion Queue
|   reorders backlog data by timestamp (handles out-of-order arrival)
+-- Redis Cache Layer
|   - live position per device (fast reads, TTL-based expiry)
|   - calibration constants per device
|   - ML correction results (keyed by input-window hash, avoids recompute)
+-- Kalman Filter (Python)
|   fuses accel + gyro + magnetometer -> rough position/orientation
+-- ML Bridge
|   checks Redis cache first -> else calls ML model -> caches result
```

```
|      (ML model itself is trained/provided by teammate)
+-- Gap Handler
|      flags any backlog segment as "reconstructed" vs real-time
+-- Database Layer
|      Postgres / TimescaleDB -- permanent trajectory history (source of truth)
+-- WebSocket Push
    sends corrected position back to Flutter app

    |
    v
```

FLUTTER APP (continued)

```
+-- Updates live map/UI
+-- Updates local cache with new corrected position
```

3. Step-by-Step Data Flow

- 1 Flutter app reads raw sensor data (accelerometer, gyroscope, magnetometer) continuously.
- 2 Every reading is saved to the local on-device cache/buffer (SQLite/Hive) immediately — before any network attempt. This guarantees no data loss even if offline.
- 3 If online: Sync Manager streams data to the backend over WebSocket in real time.
- 4 If offline: data keeps accumulating locally, AND a lightweight Kalman filter runs directly on the phone using that buffered data, so the map keeps updating live (marked as 'estimated') instead of freezing on a stale position.
- 5 On reconnect: the buffered backlog is sent to the backend in timestamp order, tagged so the backend knows it's a backlog, not live data. Once the backend's corrected result returns, it overrides the local estimate on screen.
- 6 Backend's Ingestion Queue sorts and reorders any out-of-sequence data (important after a backlog dump).
- 7 The Kalman Filter fuses the three sensor streams into a rough, noise-reduced position/orientation estimate.
- 8 The ML Bridge first checks the Redis cache to see if this exact data window was already processed; if not, it calls the ML model for a drift correction.
- 9 The correction is applied to the rough estimate, producing the final corrected position.
- 10 The corrected position is cached in Redis (for instant future reads) and stored permanently in Postgres/TimescaleDB.
- 11 The corrected position is pushed back to the Flutter app via WebSocket, which updates both the live map UI and its own local cache.

4. Offline & Failure Handling

Since dead reckoning is most useful exactly where connectivity is poor (tunnels, basements, dense buildings), the system is designed to degrade gracefully rather than fail.

Scenario	Handling
WiFi/mobile data drops mid-session	App keeps buffering locally in SQLite; a lightweight on-device Kalman filter keeps the map live (marked 'estimated'); WebSocket client auto-reconnects with exponential backoff
Backend server temporarily down	App detects failed connection, keeps buffering, retries periodically until backend responds
App killed/restarted while offline	Local buffer persists across restarts (stored on disk, not memory) — nothing is lost
Backlog arrives out of order after reconnect	Backend sorts by timestamp before running the Kalman filter, not by arrival order
Large backlog after long offline period	Backend processes backlog in smaller chunks to avoid blocking real-time users
Partial packet loss (weak signal, not full drop)	Each batch carries a sequence number; backend detects gaps and can request a resend or interpolate small gaps

4B. Local Fallback Kalman Filter (On-Device)

In the base design, the app shows a frozen 'last known position' while offline. This refinement adds a second, lightweight Kalman filter that runs directly on the phone (in Dart) using the same buffered sensor data. It has no ML drift correction — it is intentionally simple — but it means the on-screen position keeps moving in real time during a disconnection instead of freezing.

- Activates automatically only when the Connectivity Monitor detects an offline state.
- Produces an uncorrected, 'best effort' position estimate from the local sensor buffer.
- The UI clearly labels this as 'estimated' (uncorrected) to distinguish it from a backend-verified position.
- Once connectivity returns and the backend sends back the true, ML-corrected position, it overrides and replaces the local estimate.

This is a deliberately scoped-down version of a more advanced idea from a reviewed peer architecture (which proposed a full on-device Extended Kalman Filter running inside a dedicated background thread). The simplified version keeps the same user-facing benefit — a live map during disconnection — without requiring multi-threaded isolate management or a second full sensor-fusion implementation to build and debug under time pressure.

5. Caching Layers Explained

A cache is fast, temporary storage for information needed quickly and repeatedly, so the system avoids redoing slow work. This architecture uses three distinct caches, each with a different job:

Cache	Location	Purpose
Live position cache	Redis (backend)	Stores each device's most recent corrected position for instant reads; TTL-based expiry removes inactive devices automatically
ML correction cache	Redis (backend)	Stores ML model outputs keyed by a hash of the input sensor window, avoiding redundant model calls when backlog data overlaps previously processed data
Calibration cache	Redis (backend)	Stores per-device sensor calibration constants so they aren't re-fetched from Postgres on every incoming batch
Local UI cache	SQLite/Hive (Flutter app)	Stores the last known corrected position so the UI shows something meaningful (with a 'last updated' timestamp) instead of a blank screen during disconnection

Note the distinction: Redis is a fast, short-lived cache for live/recent data; Postgres/TimescaleDB is the permanent source of truth for full trajectory history; the Flutter-side local cache exists purely to keep the app usable during disconnection.

6. Technology Stack Summary

Layer	Technology
Frontend framework	Flutter / Dart

Sensor access	sensors_plus (Flutter package)
Local device storage	sqflite or Hive
Connectivity detection	connectivity_plus
Real-time communication	WebSocket (web_socket_channel)
Backend framework	Python + FastAPI
Sensor fusion (backend)	Kalman Filter (Python, e.g. filterpy library)
Sensor fusion (on-device fallback)	Lightweight Kalman Filter (Dart), active only offline
Drift correction	ML model (LSTM/GRU or similar, provided by teammate)
Fast cache	Redis
Permanent storage	PostgreSQL with TimescaleDB extension